

ANN-Based Energy Consumption Predictor

```
In [1]: # Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
In [2]: # Load Dataset
df = pd.read_csv("dataset/energy.csv")
df.head()
```

```
Out[2]:
```

	date	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2	44.790000	19.79	44.730000
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2	44.722500	19.79	44.790000
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2	44.626667	19.79	44.933333
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2	44.590000	19.79	45.000000
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2	44.530000	19.79	45.000000

5 rows x 29 columns

```
In [3]: df = df.sample(5000, random_state=42)
```

```
In [4]: print(df.columns)
Index(['date', 'Appliances', 'lights', 'T1', 'RH_1', 'T2', 'RH_2', 'T3',
      'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7', 'T8',
      'RH_8', 'T9', 'RH_9', 'T_out', 'Press_mm_hg', 'RH_out', 'Windspeed',
      'Visibility', 'Tdewpoint', 'rv1', 'rv2'],
      dtype='object')
```

```
In [5]: # DATA PREPROCESSING
```

```
# Check dataset information
print(df.info())

# Check missing values
print(df.isnull().sum())

# Remove missing values if any
df = df.dropna()

# Convert date column to datetime
df['date'] = pd.to_datetime(df['date'])

# Extract useful time features
df['hour'] = df['date'].dt.hour
df['day'] = df['date'].dt.day
df['month'] = df['date'].dt.month

# Drop original date column
df = df.drop('date', axis=1)

# =====
# FEATURES AND TARGET
# =====

# Target variable
y = df['Appliances']

# Feature variables
X = df.drop('Appliances', axis=1)

# =====
# TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# =====
# FEATURE SCALING
# =====

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Preprocessing completed successfully!")
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 5000 entries, 8980 to 14591
Data columns (total 29 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                   5000 non-null   object
1   Appliances              5000 non-null   int64
2   lights                  5000 non-null   int64
3   T1                      5000 non-null   float64
4   RH_1                    5000 non-null   float64
5   T2                      5000 non-null   float64
6   RH_2                    5000 non-null   float64
7   T3                      5000 non-null   float64
8   RH_3                    5000 non-null   float64
9   T4                      5000 non-null   float64
10  RH_4                    5000 non-null   float64
11  T5                      5000 non-null   float64
12  RH_5                    5000 non-null   float64
13  T6                      5000 non-null   float64
14  RH_6                    5000 non-null   float64
15  T7                      5000 non-null   float64
16  RH_7                    5000 non-null   float64
17  T8                      5000 non-null   float64
18  RH_8                    5000 non-null   float64
19  T9                      5000 non-null   float64
20  RH_9                    5000 non-null   float64
21  T_out                   5000 non-null   float64
22  Press_mm_hg             5000 non-null   float64
23  RH_out                  5000 non-null   float64
24  Windspeed               5000 non-null   float64
25  Visibility               5000 non-null   float64
26  Tdewpoint               5000 non-null   float64
27  rv1                     5000 non-null   float64
28  rv2                     5000 non-null   float64
dtypes: float64(26), int64(2), object(1)
memory usage: 1.1+ MB
None
date                   0
Appliances             0
lights                 0
T1                     0
RH_1                   0
T2                     0
RH_2                   0
T3                     0
RH_3                   0
T4                     0
RH_4                   0
T5                     0
RH_5                   0
T6                     0
RH_6                   0
T7                     0
RH_7                   0
T8                     0
RH_8                   0
T9                     0
RH_9                   0
T_out                  0
Press_mm_hg            0

```

```

RH_out      0
Windspeed   0
Visibility   0
Tdewpoint   0
rv1          0
rv2          0
dtype: int64
Preprocessing completed successfully!
Training data shape: (4000, 30)
Testing data shape: (1000, 30)

```

```

In [6]: # ANN MODEL TRAINING

from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Create ANN model
model = MLPRegressor(
    hidden_layer_sizes=(32,16),
    activation='relu',
    solver='adam',
    max_iter=100,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

print("Model training completed!")

# MODEL EVALUATION

# Make predictions
y_pred = model.predict(X_test)

# Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Mean Absolute Error:", mae)
print("Root Mean Squared Error:", rmse)

# VISUALIZATION

plt.figure(figsize=(10,6))

plt.plot(y_test.values[:100], label='Actual Values')
plt.plot(y_pred[:100], label='Predicted Values')

plt.xlabel("Samples")
plt.ylabel("Energy Consumption")
plt.title("Actual vs Predicted Energy Consumption")

plt.legend()
plt.show()

```

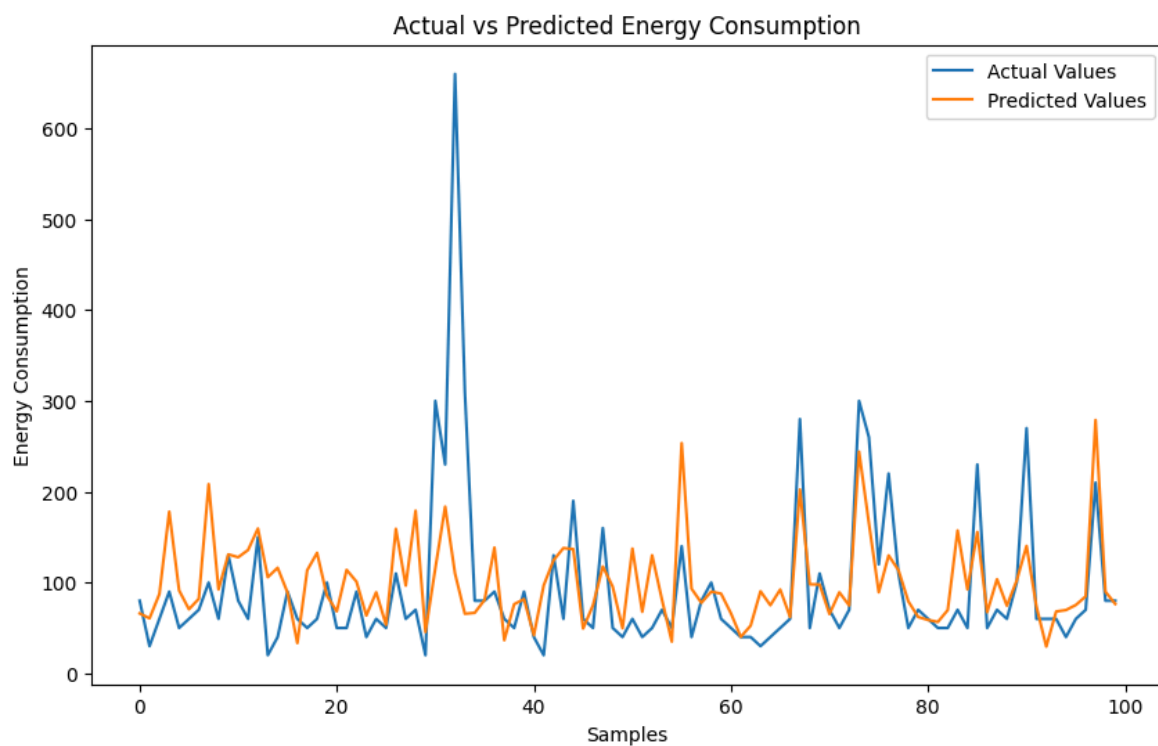
```
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:780: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (100) reached and the optimization hasn't converged yet.
```

```
warnings.warn(
```

Model training completed!

Mean Absolute Error: 51.964084205876844

Root Mean Squared Error: 92.74270343499848



In []: